

x86 Processor Memory Management

Course Teacher:

Md. Obaidur Rahman, Ph.D.

Professor

Department of Computer Science and Engineering (CSE)

Dhaka University of Engineering & Technology (DUET), Gazipur.

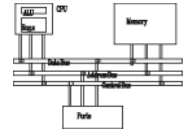
Course ID: CSE - 4503

Course Title: Microprocessors and Assembly Language

Department of Computer Science and Engineering (CSE),

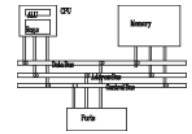
Islamic University of Technology (IUT), Gazipur.

Lecturer Reference

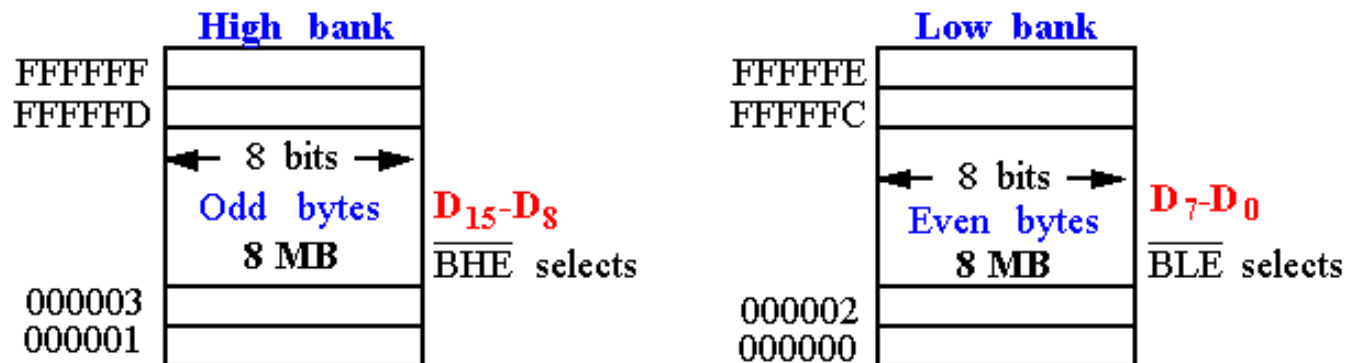


- Lecture Material:
 - Intel x86 Architecture: *Computer Organization and Assembly Languages*, Yung-Yu Chuang
 - *with slides by Kip Irvine*

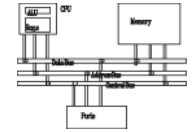
Memory Banks of 8086



- ODD Bank and EVEN BANK
- If a 16-bit data is stored in a memory location started with even address and ends at odd address, then the 8086 processor can read the data in one cycle.
- If a 16-bit data is stored in a memory location started with a odd address and ends at even address, then the 8086 processor can read the data in two cycles.
- If 8-bit data is stored in either even or odd memory location then the 8086 processor can read it in one cycle.

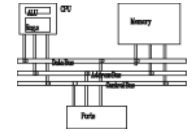


Real-address mode (in 8086)

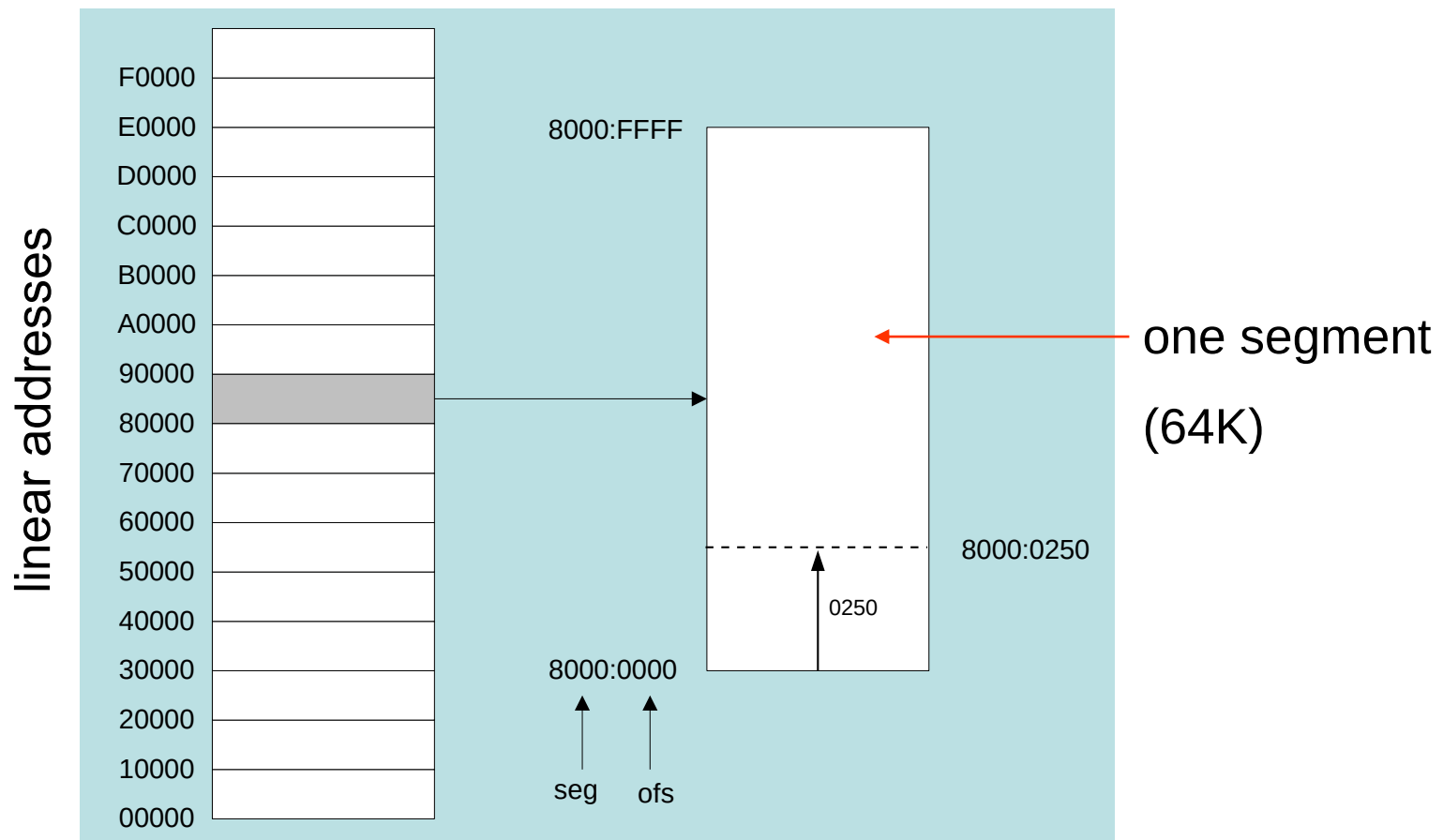


- 1 MB RAM maximum addressable (20-bit address)
- Application programs can access any area of memory
- Single tasking
- Supported by MS-DOS operating system

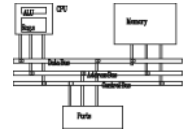
Real-address Mode: Segmented memory



Segmented memory addressing: absolute (linear) address is a combination of a 16-bit segment value added to a 16-bit offset



Real-address Mode: Calculating linear addresses

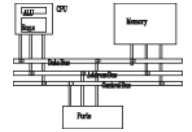


- Given a segment address, multiply it by 16 (add a hexadecimal zero), and add it to the offset
- Example: convert 08F1:0100 to a linear address

Adjusted Segment value:	0	8	F	1	0		
Add the offset:			0	1	0	0	
Linear address:			0	9	0	1	0

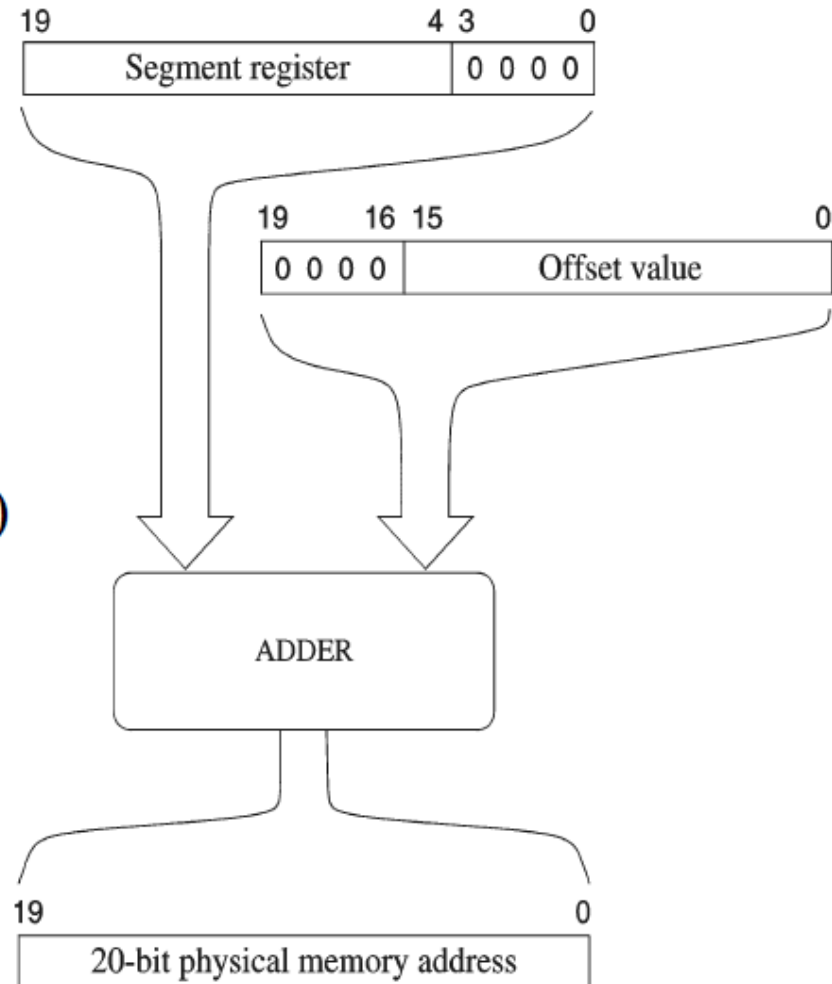
- A typical program has three segments: code, data and stack. Segment registers CS, DS and SS are used to store them separately.

Real-address Mode: Calculating linear addresses

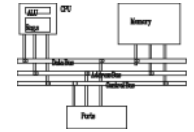


- Conversion from logical to physical addresses

$$\begin{array}{r} 11000 \text{ (add 0 to base)} \\ + \quad 450 \text{ (offset)} \\ \hline 11450 \text{ (physical address)} \end{array}$$



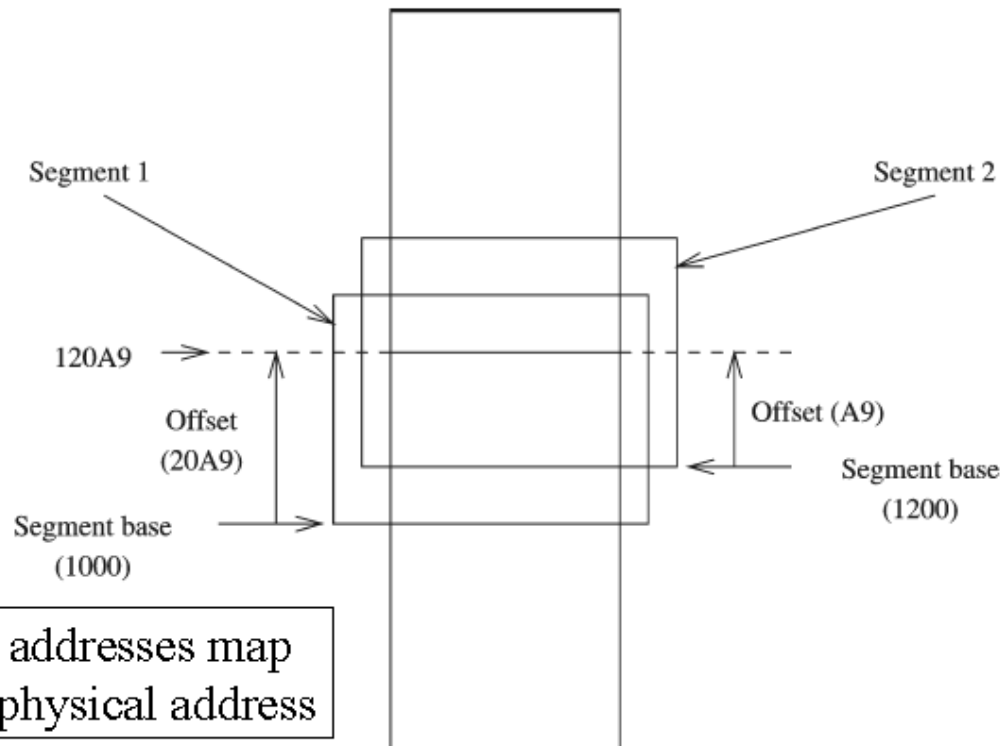
Real-address Mode: Example



What linear address corresponds to the segment/offset address 028F:0030?

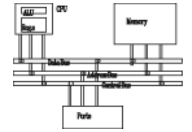
$$028F0 + 0030 = 02920$$

Always use hexadecimal notation for addresses.



Two logical addresses map to the same physical address

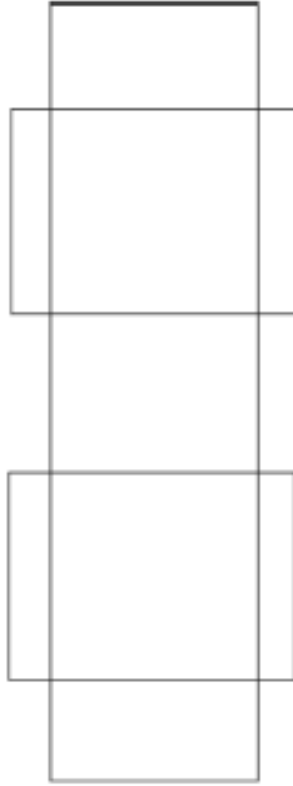
Real-address Mode: Example



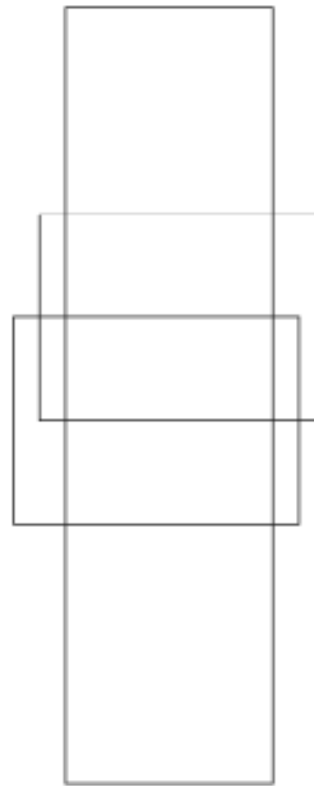
- Segment Overlapping



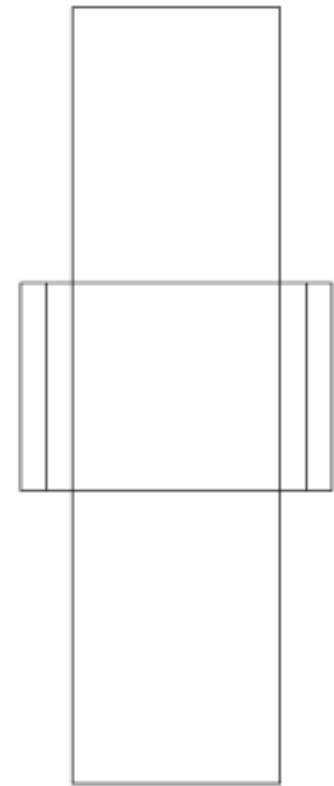
(a) Adjacent



(b) Disjoint



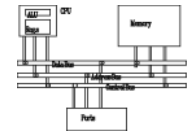
(c) Partially overlapped



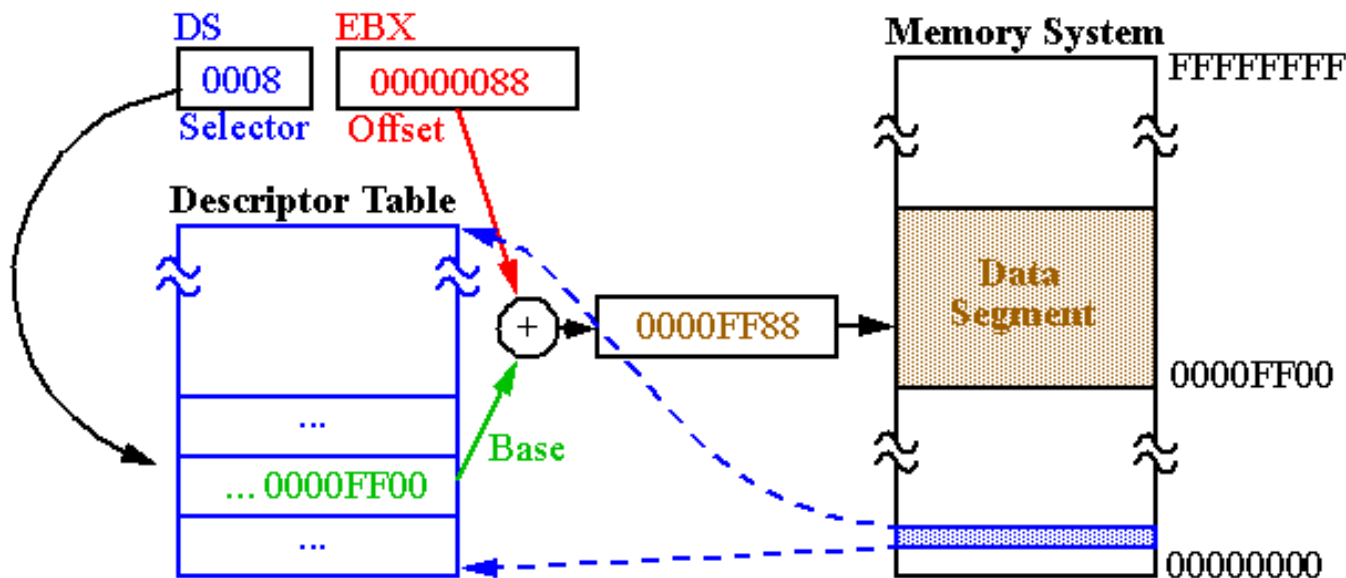
(d) Fully overlapped

EMU 8086 uses **Fully Overlapped Segment**

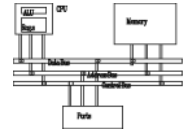
Protected mode (in 80286 and Upper)



- 80286 Processor with 16 MB addressable RAM (24-bit address bus) (000000 to FFFFFFFh)
- 80386 and upper Processors with 4 GB addressable RAM (32-bit address bus) (00000000 to FFFFFFFFh)
- Each program assigned with a memory partition which is protected from other programs
- Designed for multitasking
- Supported by Linux & MS-Windows

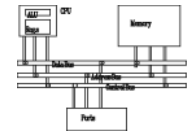


Protected mode



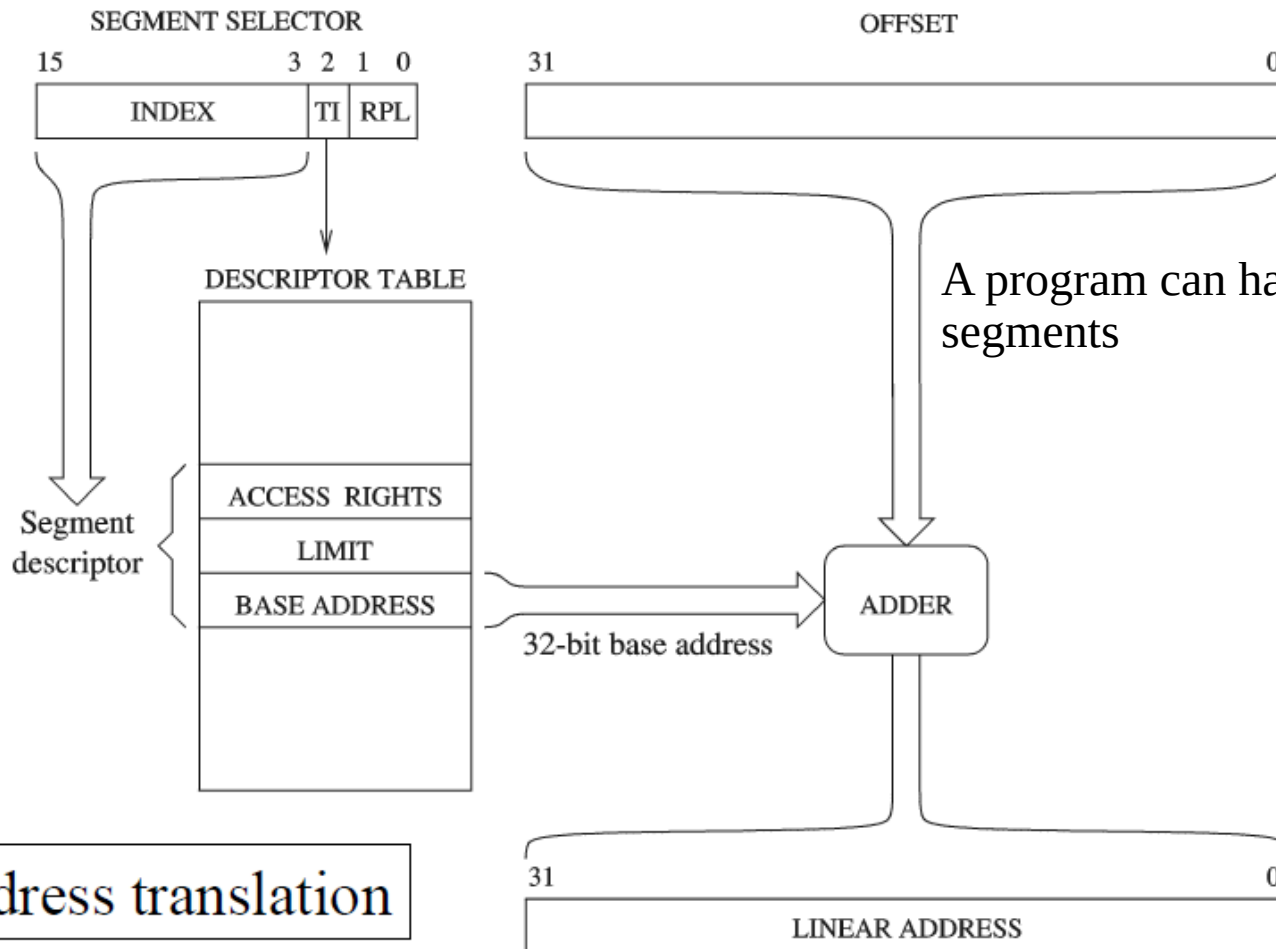
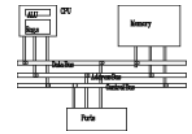
- Started **with 80286 up to Pentium**, all processors use 2 modes for memory address management
 - * Real mode
 - » Uses 16-bit addresses
 - » Runs 8086 programs
 - » Pentium acts as a **faster 8086**
 - * Protected mode
 - » 32-bit mode
 - » **Native mode of Pentium**
 - » Supports **segmentation and paging**

Protected Mode (Types of Address Space)



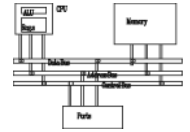
- **Logical/Virtual Address:**
 - An address used by application program consists of **16-bit selector** and **32-bit offset value**.
 - In a flat memory model, 16-bit selector is loaded in CS, DS, SS and ES registers and **finds the 32-bit base address**.
- **Linear Addresses:**
 - It is calculated from virtual address by **segmentation unit**.
 - The base of the segment refer to by the selector is added with the **32-bit offset value**, generating a 32-bit linear address.
 - **When paging is disabled, Linear Address = Physical Address**
- **Physical Address:**
 - The address value appears on the address bus of a microprocessor during a memory read/write operation
 - Physical address calculated from linear addresses **through paging unit**.
 - Linear address is **used as an index** into the page table, where the CPU locates the corresponding physical address.

Protected Mode in 80386: Calculating Linear Addresses



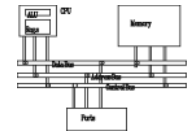
Address translation

Protected Mode: Calculating Addresses

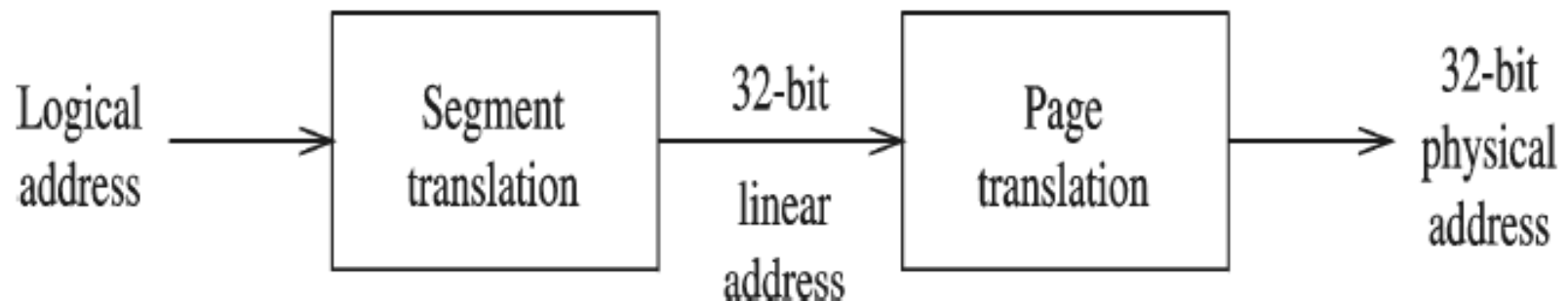


- Index
 - * Selects a descriptor from one of two descriptor tables
 - » Local
 - » Global
- Table Indicator (TI)
 - * Select the descriptor table to be used
 - » 0 = Local descriptor table
 - » 1 = Global descriptor table
- Requestor Privilege Level (RPL)
 - * Privilege level to provide protected access to data
 - » Smaller the RPL, higher the privilege level

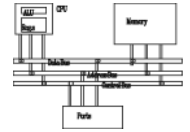
Protected mode



- Supports **sophisticated segmentation**
- Segment unit translates 32-bit logical address to 32-bit linear address
- Paging unit translates 32-bit linear address to 32-bit physical address
 - * If no paging is used
 - » Linear address = physical address

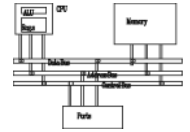


Protected mode



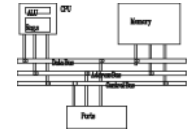
- In this mode there is a Segment Descriptor Table
- **Typical Program structure** follows:
 - Code, Data, and Stack areas
 - CS, DS, SS segment descriptors
 - Global Descriptor Table (GDT)
 - Local Descriptor Table (LDT)

Protected mode (Address Translation)

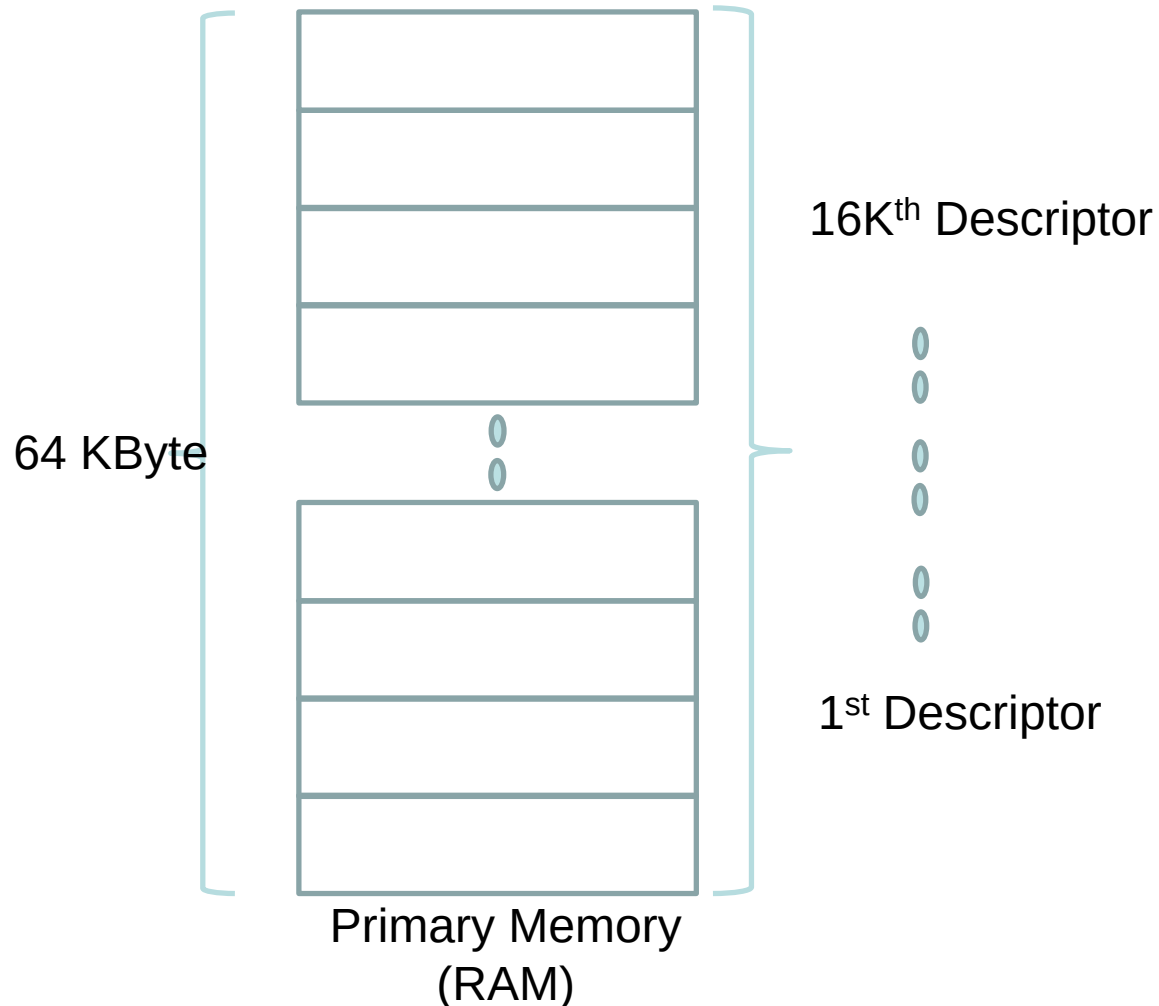


- 80386 and later microprocessors transform logical address into physical address in 2 steps:
 - Segment translation
 - Page translation
- These translations are done in a way that it is not visible to the application programmers or users

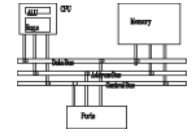
Descriptor Table (For 80286, NOT



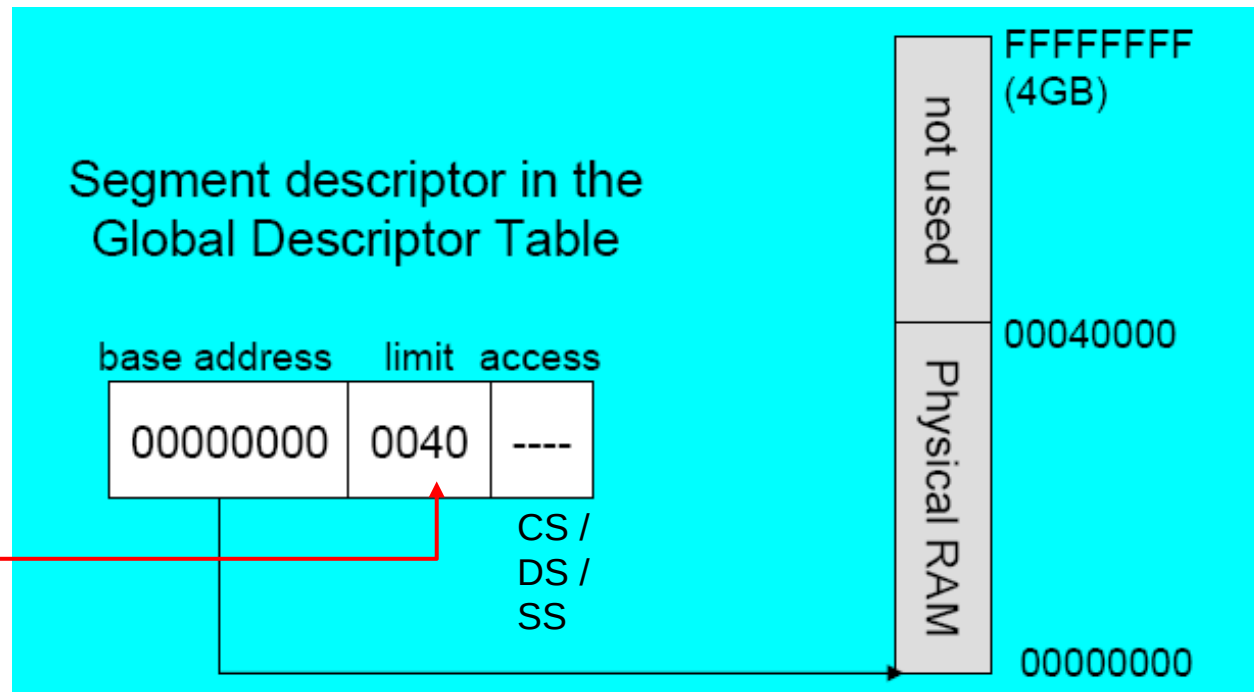
- In protected mode, the concept of **Segment** is not used.
- A virtual address consists of a 16-bit **Selector** and 16-bit **Offset**.
- A memory management unit (MMU) uses 16-bits of selector to access a **Descriptor** for the desired segment in a table of descriptors.



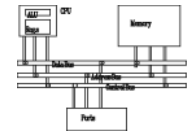
Flat Segmentation Model in 80386



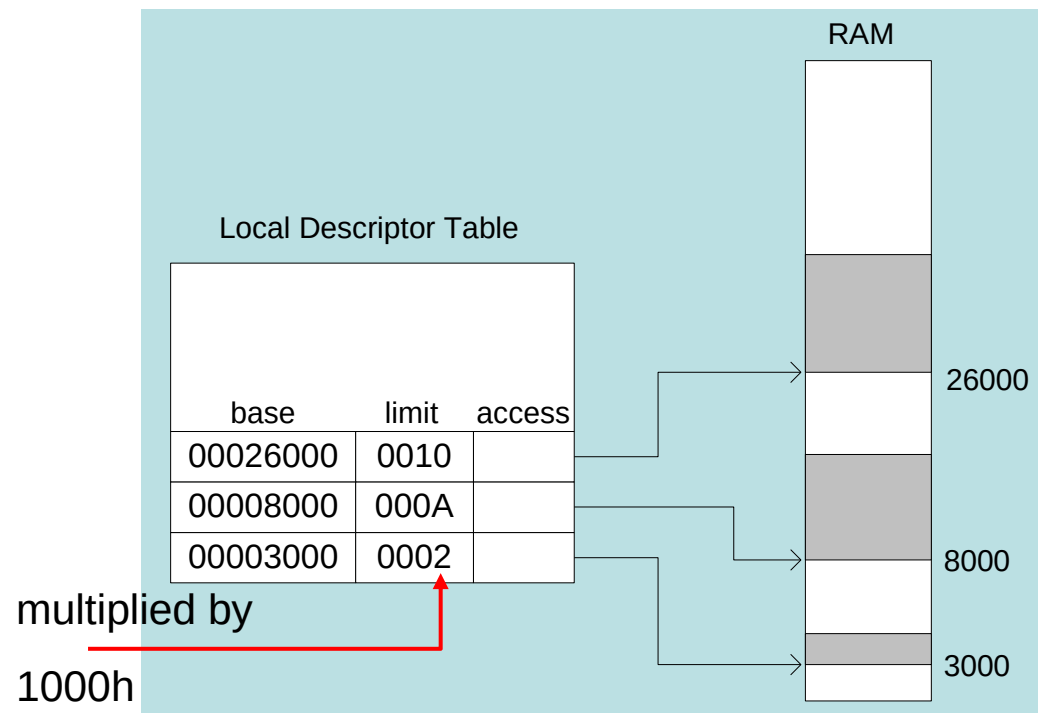
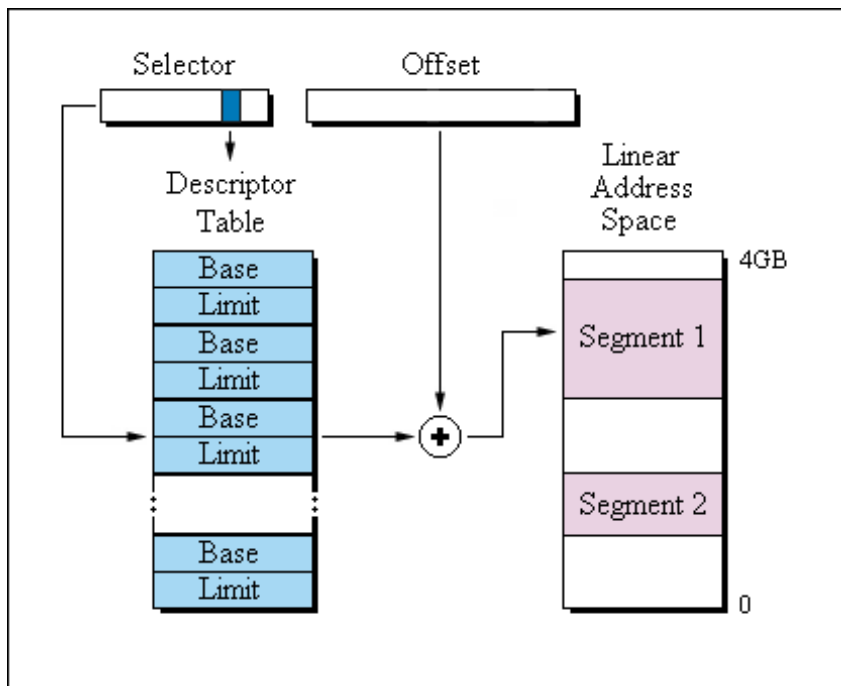
- All segments are mapped to the entire 32-bit physical address space, at least two, one for data and one for code.
- All tasks can share the GDT memory.
- GDTR Register is 48-bit. LDTR is 32-bit, of which 16-bit for segment limit and 16-bit for segment selector (CS/DS/SS). The segment selector further partitioned into 13-bits to define 8192 Descriptors, each having 8 bytes size (i.e., total 64-bits = 32-bit base address + 16-bit limit + 16-bit access). So, total memory size of GDT = 8K x 8 bytes = 64 KB.
- Global Descriptor Table (GDT)



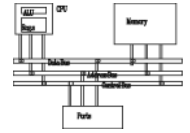
Multi-segment model



- Each program/task has a local descriptor table (LDT)
 - Holds descriptor for each segment used by the program
 - Uses any of the Descriptor of 8K GDT as LDT. So, LDT is not an independent memory table from GDT.
 - LDTR further can have max 8K Descriptors from the memory.

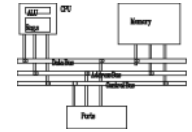


Translating Addresses

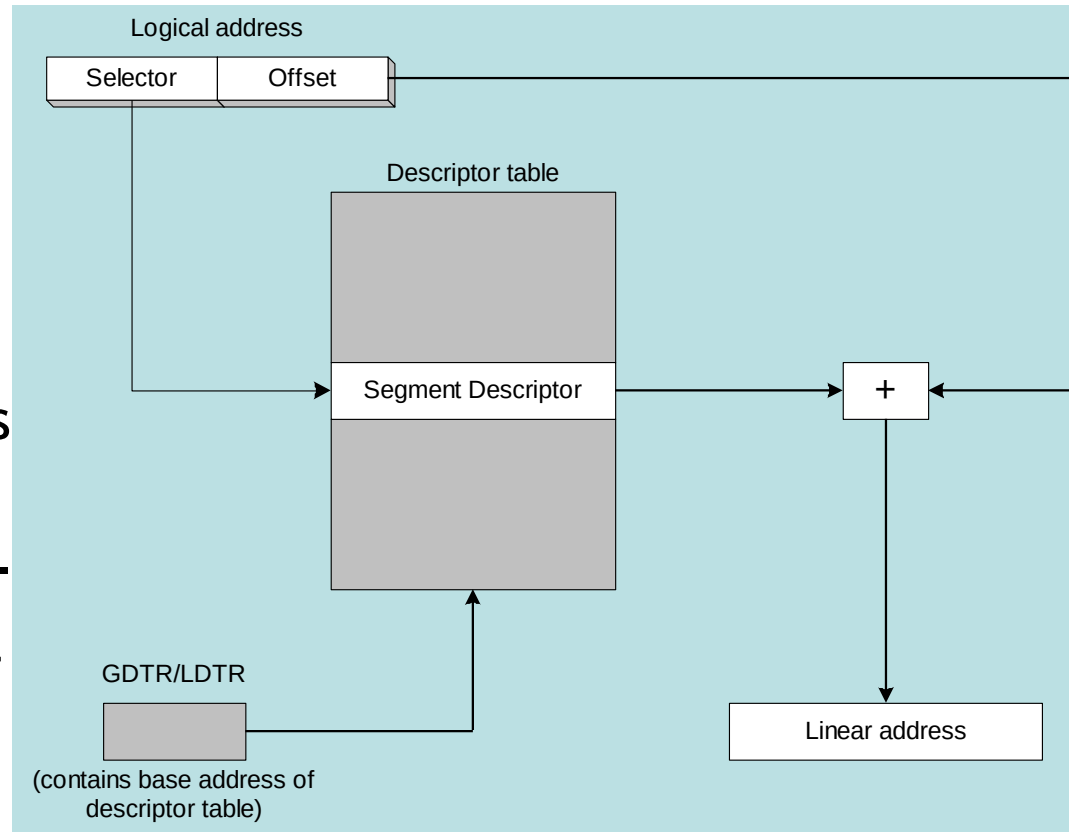


- The processor uses a one- or two-step process to convert a variable's logical address into a unique memory location.
- The first step combines a segment value with a variable's offset to create a **linear address**.
- The second optional step, called **page translation**, converts a linear address to a **physical address**.

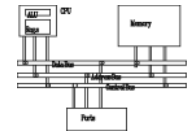
Converting Logical to Linear Address



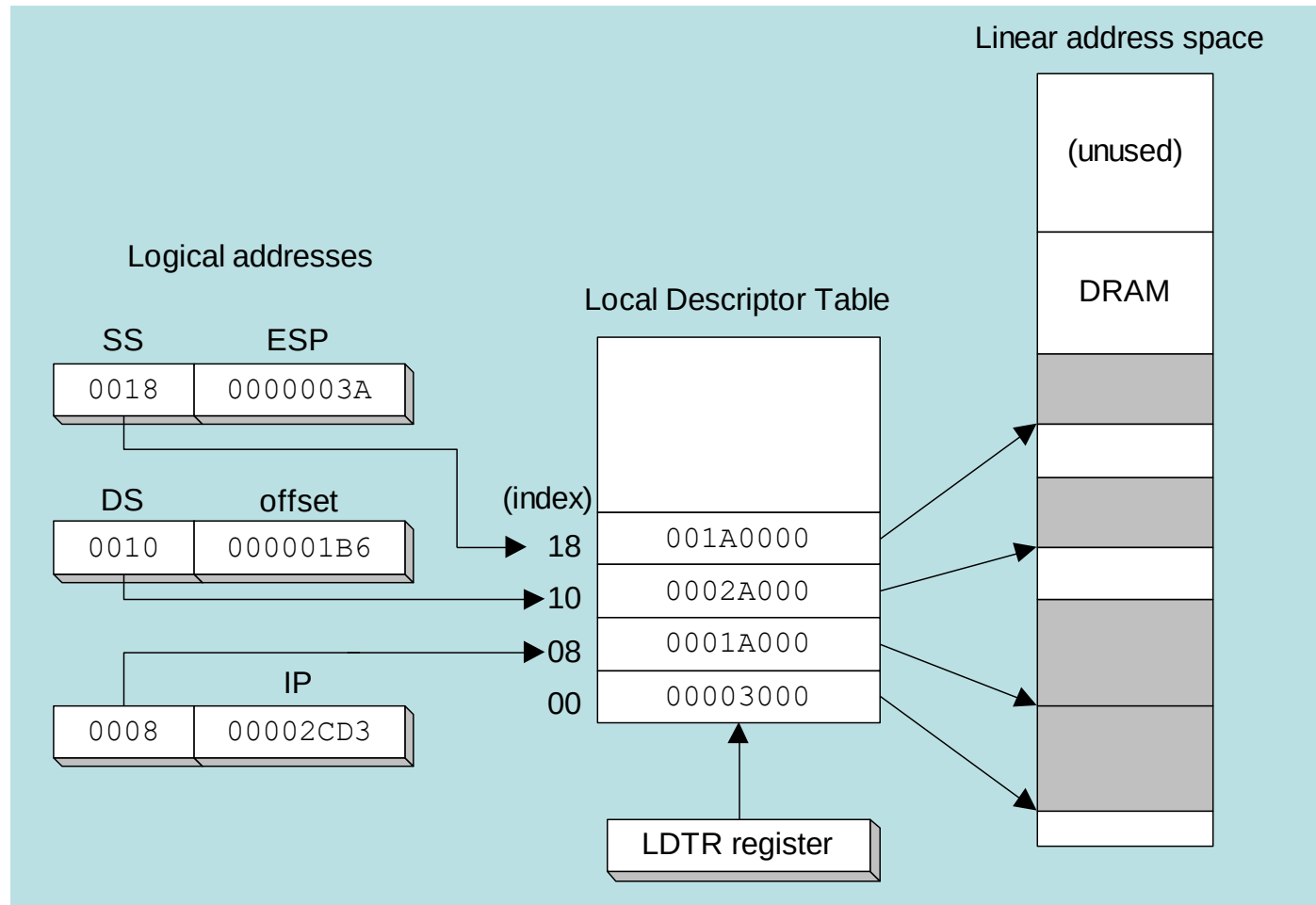
- All information about a segment is stored in a **8-byte data structure** called segment descriptor.
- The **segment selector** points to a segment descriptor, which contains the base address of a memory segment. The **32-bit offset** from the logical address is added to the segment's base address, generating a 32-bit linear address.



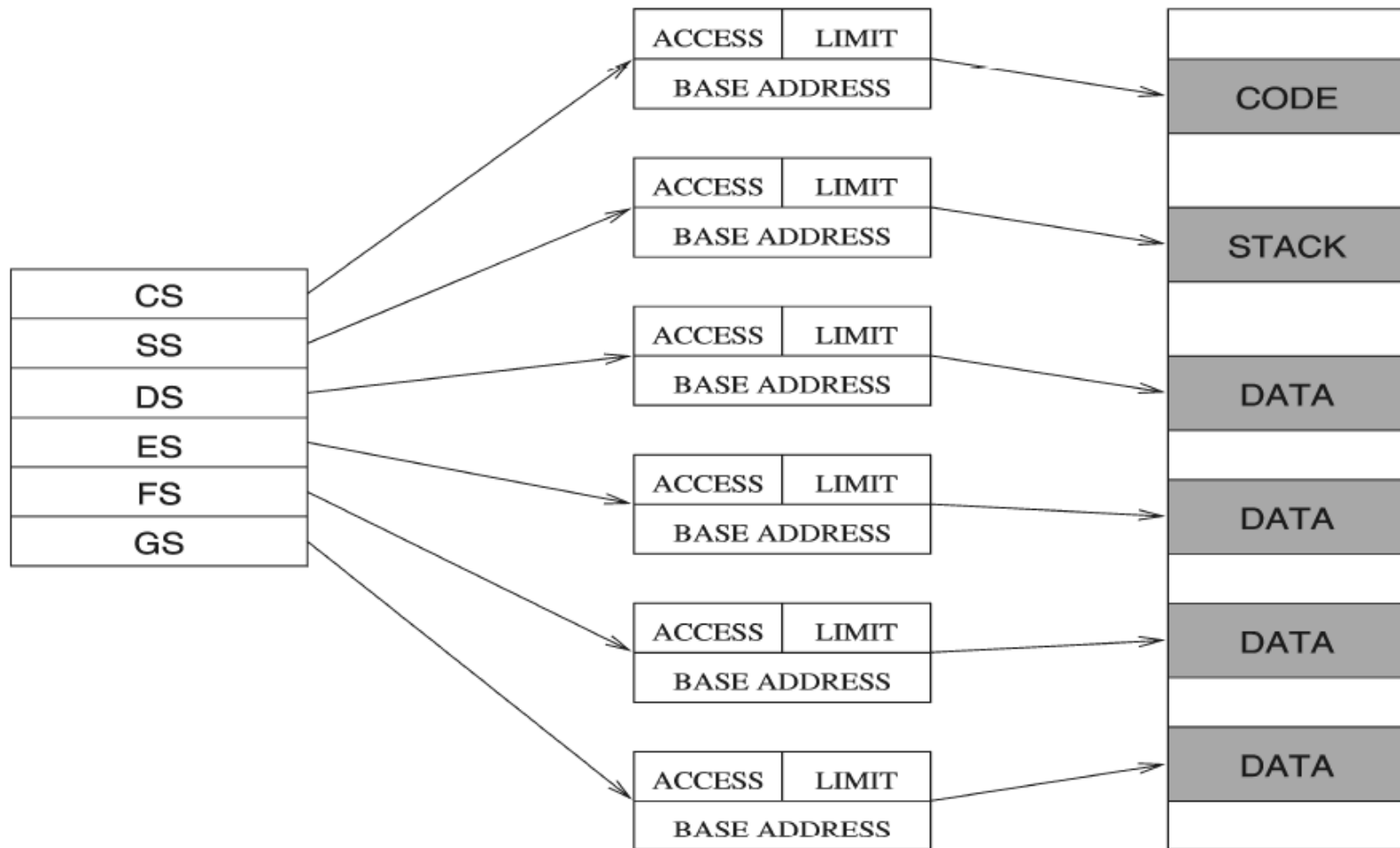
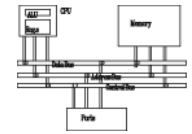
Indexing into a Descriptor Table



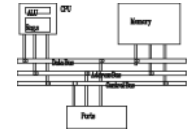
Each segment descriptor indexes into the program's local descriptor table (LDT). Each table entry is mapped to a linear address:



Finally Defining Multi-Segments

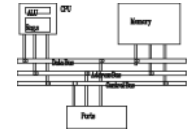


Paging



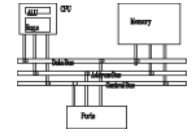
- Virtual memory uses **disk** as part of the memory, thus allowing **sum of all programs** can be larger than physical memory
- Only part of a program must be kept in **memory**, while the remaining parts are kept on **disk**.
- The **memory used by the program** is divided into small units called pages (4096-byte or 4KByte).
- As the program runs, the **processor selectively unloads inactive pages from memory and loads other pages** that are immediately required.

Paging

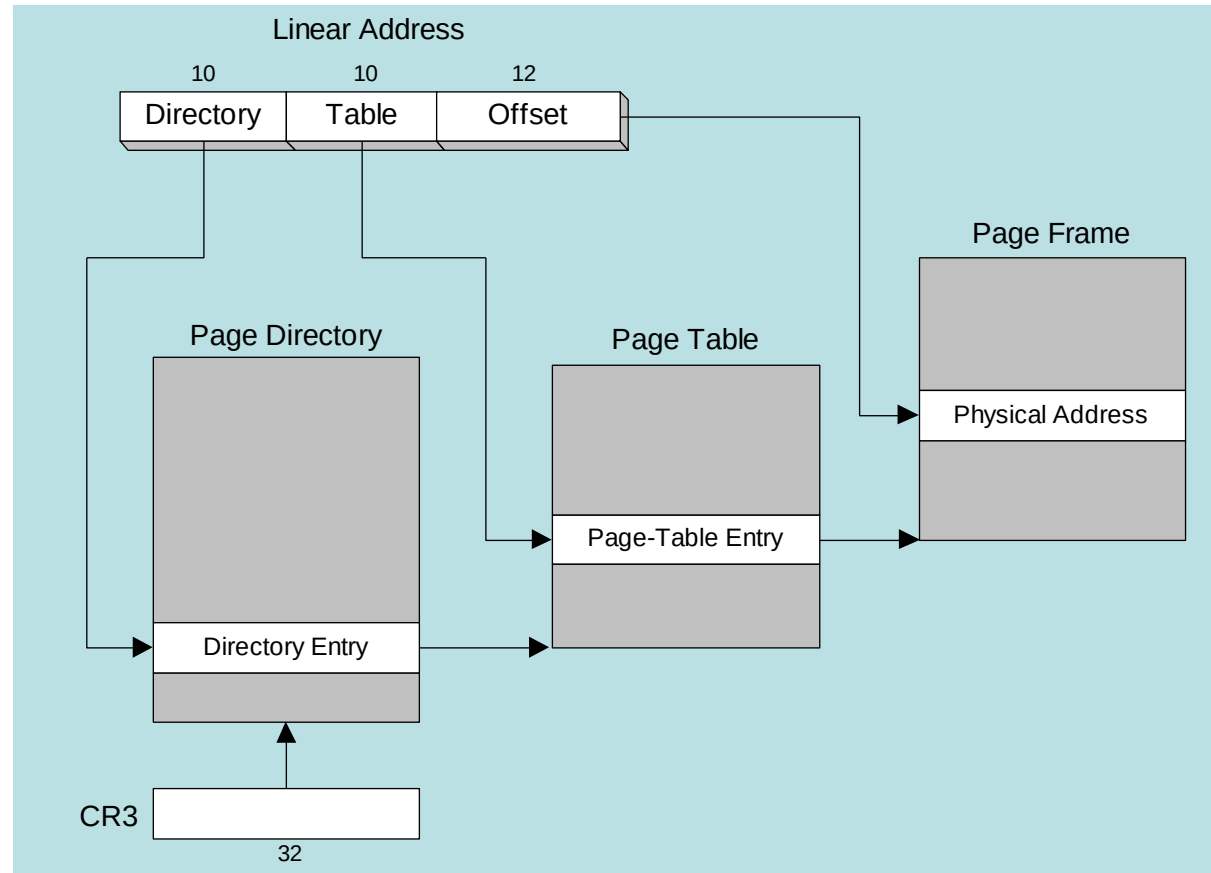


- OS maintains **page directory** and **page tables**
- **Page Translation:** CPU converts the linear address into a physical address
- **Page Fault:** Occurs when a needed page is not in memory, and the CPU interrupts the program
- **Virtual Memory Manager (VMM):** OS utility that manages the loading and unloading of pages
- OS copies the page into memory, program resumes execution

Page Translation



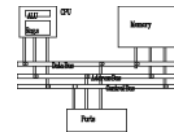
A linear address is divided into a page directory field, page table field, and page frame offset. The CPU uses all three to calculate the physical address.



The diagram illustrates a computer system architecture. At the top left is a box labeled 'CPU' containing 'ALU' and 'Register'. To its right is a box labeled 'Memory'. Below these is a horizontal line representing the system bus, which is divided into a 'Data Bus' (top) and a 'Control Bus' (bottom). The CPU is connected to the Data Bus and the Control Bus. The Memory is connected to the Data Bus. At the bottom is a box labeled 'Peripherals' connected to both the Data Bus and the Control Bus. An 'Address Bus' is also shown, connecting the CPU to the Memory.

PAGEING MECHANISM OF 80386





Thank You